

```

In[1]:= Print["Wolfram Language ready!"]
Print[$Version]

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)

In[3]:= (* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)
(* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)
(* *)
(* When V[t] ≥ Vthresh, neuron spikes and resets *)

In[4]:= (* ===== Neuron Parameters ===== *)

(* WhenEvent is a powerful Wolfram feature that lets you say:

    Perfect for spike detection and reset. *)

(* Step 3: Define Neuron Parameters *)

tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ===== Input Current ===== *)

(* Constant current that turns on at t=0.05 and off at t=0.4 *)

Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 120.0, 12.0];

Print["Parameters defined."]

Parameters defined.

```

1. The input current is defined as a function of time using `If[]`. In plain English, what does `Iinput[t]` return at $t = 0.03$? At $t = 0.2$? At $t = 0.5$? At $t = 0.03$, the condition $0.05 \leq t \leq 0.4$ is false, so `Iinput[t]` returns 0.0.

At $t = 0.2$, the condition is true because 0.2 is between 0.05 and 0.4, so `Iinput[t]` returns 120.0. At $t = 0.5$, the condition is false because 0.5 is greater than 0.4, so `Iinput[t]` returns 0.0. In simple terms, the neuron only receives input current during the time interval 0.05 to 0.4 seconds.

2. Why do we use $:=$ (delayed assignment) for `Iinput` instead of $=$ (immediate assignment)? We use delayed assignment so that the function `Iinput[t]` is evaluated each time it is called with a specific value of t . This allows Wolfram to check the condition $0.05 \leq t \leq 0.4$ dynamically while the simulation runs. Therefore, if we used immediate assignment instead, the expression would be evaluated only once when it is defined, which could prevent the function from correctly responding to different values of t during the simulation. Using delayed assignment ensures that the input current changes correctly over time during the neuron simulation.

```
In[11]:= (* Solve the LIF equation with spike reset *)
```

```
spikeCount = 12
;
spikeTimes = {};

sol = NDSolve[
{
tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],

V[0] == Vrest,

WhenEvent[V[t] ≥ Vthresh,
{
V[t] → Vreset,
spikeCount++,
AppendTo[spikeTimes, t]
}}
],

V,
{t, 0, 0.5},
DiscreteVariables → {spikeCount}
];

Print[, spikeCount,];
Print[, spikeTimes];

Neuron fired 12 spikes

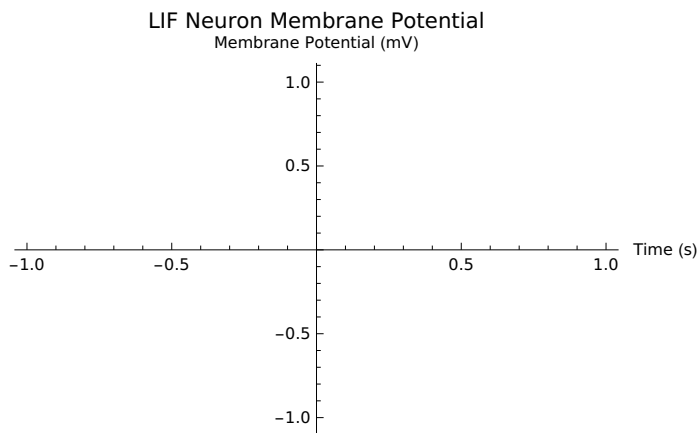
Spike times: {}
```

```

In[12]:= (Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  PlotRange -> All,
  AxesLabel -> {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel -> "LIF Neuron Membrane Potential",
  Epilog -> {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
])

```

Out[12]=



1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?

- The membrane potential V between spikes forms a curve, not a straight line. This happens because the LIF model follows a differential equation that produces exponential behavior. As input current increases the voltage, the leak term slows the rate of increase as V approaches the threshold. Because of this leak effect, the voltage rises smoothly in a curved shape rather than increasing linearly.

2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

- After $t > 0.4$, the input current becomes zero, so the neuron no longer receives stimulation. As a result, the membrane potential V gradually decreases back toward the resting potential. According to the LIF equation, only the leak term remains active, causing the voltage to decay exponentially. This means the neuron stops firing spikes and slowly returns to its resting state.

```

In[13]:= Manipulate[
  Module[{sol, sc = 0, st = {}, iFunc},

    (* Local input current function *)
    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, 2*currentVal, 0.0];

    (* Solve LIF neuron *)
    sol = NDSolve[
      {
        tauVal*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
        V[0] == Vrest,

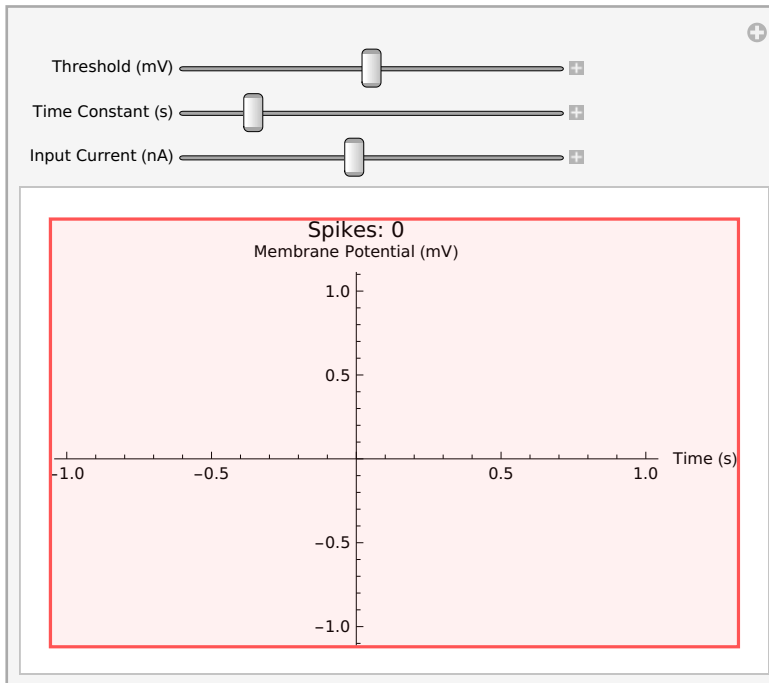
        WhenEvent[V[t] ≥ thresh,
          {V[t] → Vreset, sc++, AppendTo[st, t]}]
      },
      V,
      {t, 0, 0.5},
      DiscreteVariables → {sc}
    ];

    (* Plot membrane potential *)
    Plot[
      Evaluate[V[t] /. sol],
      {t, 0, 0.5},
      PlotRange → All,
      AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
      PlotLabel → StringForm["Spikes: ``", sc],
      Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
    ]
  ],

  (* Sliders *)
  {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
  {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
  {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]

```

Out[13]=



1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

- Using the dashboard with the default threshold (-55 mV) and time constant (0.020 s), I slowly increased the input current until the neuron produced at least one spike. The neuron first fired a spike at approximately 20 nA. This value represents the rheobase, which is the minimum input current required to push the membrane potential to the threshold and trigger a spike.

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

- With the current set to 25 nA, increasing the threshold from -65 mV toward -45 mV eventually stops the neuron from firing. The neuron stopped firing at approximately -48 mV. This happens because a higher threshold requires a larger change in membrane potential to trigger a spike. When the threshold becomes too high, the input current is not strong enough to push the voltage to that level.

3. Set threshold to -55 and current to 20 . Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more spikes? Which produces a smoother membrane potential curve?

- With threshold set to -55 mV and current at 20 nA, the neuron produced more spikes when $\tau = 0.010$. A smaller time constant allows the membrane potential to change more quickly, so it reaches the threshold more often. When $\tau = 0.050$, the membrane potential changes more slowly, producing fewer spikes but a smoother voltage curve because the voltage evolves more gradually over time.

```

In[14]:= (* Generate f-I curve data *)

fiData = Table[
Module[{spikes, iFunc, sol},

iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0];

spikes = Reap[
NDSolve[
{
tau * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
V[0] == Vrest,

WhenEvent[V[t] ≥ Vthresh,
{V[t] → Vreset, Sow[t]}
]
},
V,
{t, 0, 0.5}
]
][[2]];

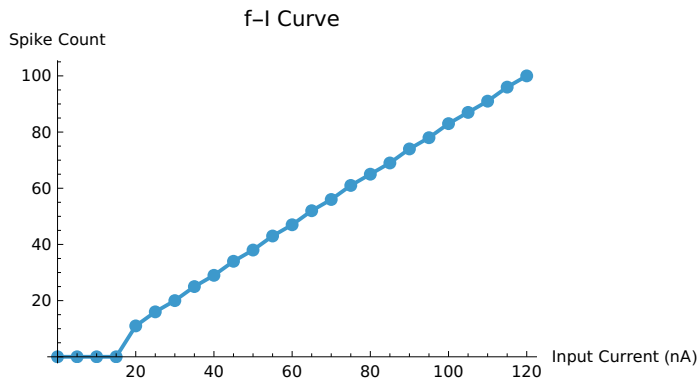
{Icurr, Length[Flatten[spikes]]}
],

{Icurr, 0, 120, 5}
];

ListLinePlot[
fiData,
AxesLabel → {"Input Current (nA)", "Spike Count"},
PlotLabel → "f-I Curve",
PlotMarkers → Automatic
]

```

Out[15]=



1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

- From the graph, the neuron begins firing at approximately 20 nA of input current. Below this value, the spike count remains at zero, meaning the membrane potential never reaches the firing threshold. This value corresponds to the rheobase, which is the minimum current required to trigger at least one spike in the neuron.

2. Is the f-I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

- The f-I curve appears mostly linear (a straight line) once the neuron begins firing. This means that as input current increases, the spike count increases at a roughly constant rate. A perfectly straight line would indicate that the neuron encodes stimulus intensity linearly, meaning firing rate increases proportionally with the strength of the input signal.

3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

- The LIF model in this simulation did not show saturation at high currents. Instead, the firing rate continues to increase as the input current increases. This occurs because the simple LIF model lacks biological mechanisms such as refractory periods or ion channel limits, which in real neurons prevent firing rates from increasing indefinitely and cause firing rates to eventually plateau.

In[16]:=

```
(* Define a sine wave input current *)
```

```
(* The sine wave oscillates, so we shift it to be always positive *)
```

```
Isine[t_] := 15.0 + 12.0*Sin[2 Pi*5*t];
```

```
(* This gives a 5 Hz sine wave centered at 15 nA, ranging from 3 to 27 nA *)
```

```
(* TODO 7: Run NDSolve with Isine[t] as the input current *)
```

```
(* Use the same LIF equation and WhenEvent as Part A *)
```

```
(* Simulation time: 0 to 1.0 seconds *)
```

```

Vth = -50;
Vreset = -65;
Vrest = -65;
Rm = 1;
tau = 0.02;

solSine = NDSolve[
{
  V'[t] ==  $-(V[t] - Vrest) + Rm * Isine[t] / tau$ ,
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vth, V[t] → Vreset]
},
V,
{t, 0, 1.0}
];

```

```

(* TODO 8: Create a two-panel plot *)
(* Top: membrane potential V[t] *)
(* Bottom: the input current Isine[t] *)

```

```

plot1 = Plot[
  Evaluate[V[t] /. solSine],
  {t, 0, 1.0},
  PlotLabel → ,
  AxesLabel → {, },
  PlotRange → All
];

```

```

plot2 = Plot[
  Isine[t],
  {t, 0, 1.0},
  PlotLabel → "Input Current Isine[t]",
  AxesLabel → {"Time (s)", "I (nA)"},
  PlotRange → All
];

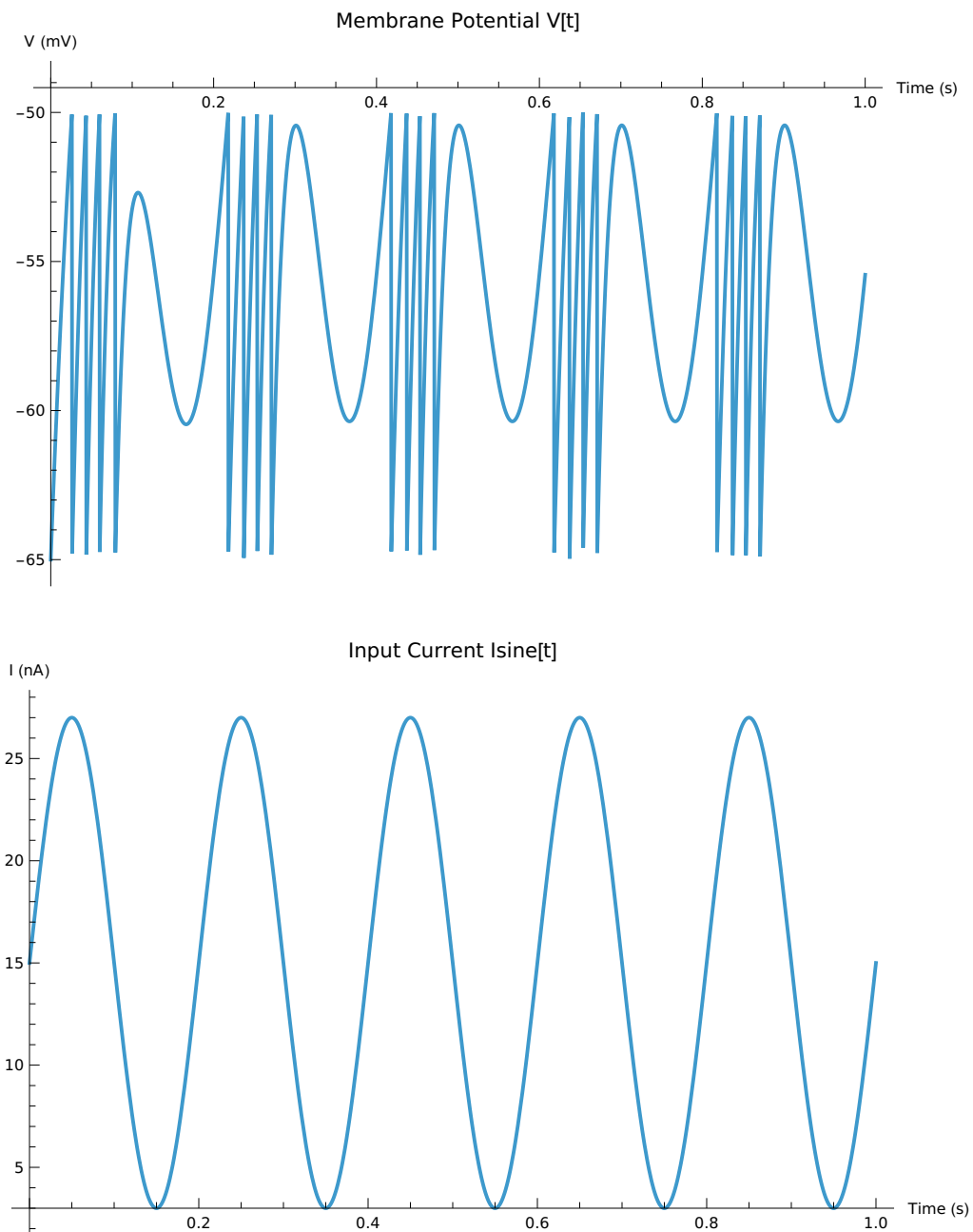
```

```

GraphicsColumn[{plot1, plot2}]

```


Out[25]=



Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

- The spike train follows the rhythm of the sine wave fairly well. When the input current increases (the peaks of the sine wave), the neuron fires many spikes close together. When the input current decreases (the valleys), the spikes become less frequent or stop. This means the timing and frequency of spikes represent the strength of the input signal.

- The information preserved is the overall pattern of the signal: higher current produces higher spike frequency and lower current produces fewer spikes. This allows the neuron to encode the signal through rate coding (spike frequency over time).
- However, some information will be lost in the conversion. The spikes do not preserve the exact continuous shape or amplitude of the sine wave. Instead of smooth voltage changes, the signal becomes discrete spike events. Fine details of the waveform and exact voltage values are therefore not retained.

EXPERIMENT 1 BELOW: Using your Manipulate[] dashboard, systematically change the threshold from -65 to -45 in steps

of 5. For each setting, record the spike count. Create a table or plot of threshold vs. spike count.

```
In[26]:= thresholds = Range[-65, -45, 5];

spikeCounts = Table[
  Module[{sol, spikes = 0},

    sol = NDSolve[
      {
        V'[t] == -(V[t] - Vrest) + Rm*Isine[t]/tau,
        V[0] == Vrest,
        WhenEvent[V[t] ≥ th, {spikes++, V[t] → Vreset}]
      },
      V,
      {t, 0, 1}
    ];

    {th, spikes}
  ],
  {th, thresholds}
];
```

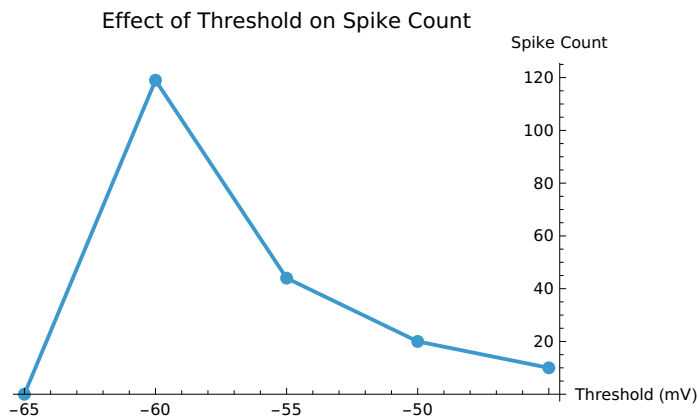
```
In[28]:= Grid[
  Prepend[spikeCounts, {"Threshold (mV)", "Spike Count"}],
  Frame → All
]
```

Out[28]=

Threshold (mV)	Spike Count
-65	0
-60	119
-55	44
-50	20
-45	10

```
In[29]:= ListPlot[
  spikeCounts,
  Joined → True,
  PlotMarkers → Automatic,
  AxesLabel → {"Threshold (mV)", "Spike Count"},
  PlotLabel → "Effect of Threshold on Spike Count"
]
```

Out[29]=



Experiment 5 Below: Two Different f-I Curves

Generate f-I curves for two different time constants (e.g., $\tau = 0.010$ and $\tau = 0.040$) and plot them on the same graph. How does τ affect the neuron's sensitivity to input current?

- The time constant (τ) controls how quickly the membrane potential responds to input current. If it is smaller, the membrane charges faster, so the neuron reaches threshold quickly and fires more spikes for the same input current. If larger, the membrane changes more slowly, so it takes longer to reach threshold, producing lower firing rates. Therefore a smaller τ makes the neuron more sensitive to input current, producing higher firing rates. A larger τ reduces sensitivity, requiring stronger input currents to produce the same firing rate.

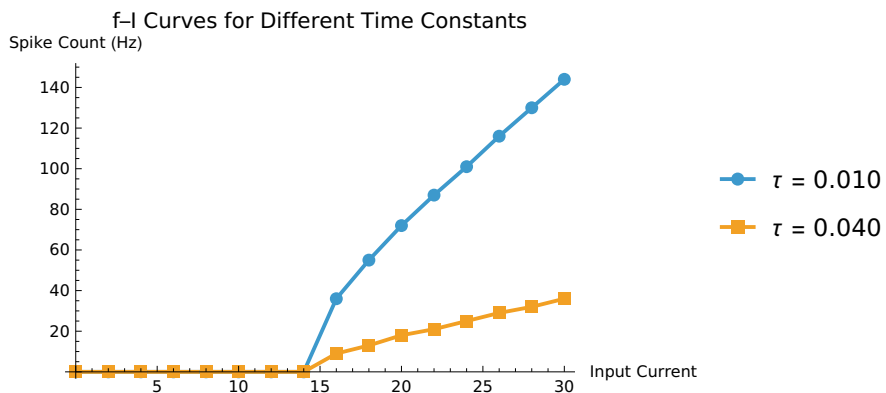
```
In[30]:= currents = Range[0, 30, 2];
```

```
spikeRate[tauValue_, I_] := Module[{spikes = 0},
  NDSolve[
    {
      V'[t] == -(V[t] - Vrest) + Rm*I)/tauValue,
      V[0] == Vrest,
      WhenEvent[V[t] ≥ Vth, {spikes++, V[t] → Vreset}]
    },
    V,
    {t, 0, 1}
  ];
  spikes
]
```

```
dataTau1 = Table[{I, spikeRate[0.010, I]}, {I, currents}];
dataTau2 = Table[{I, spikeRate[0.040, I]}, {I, currents}];
```

```
ListLinePlot[
  {dataTau1, dataTau2},
  PlotLegends → {"τ = 0.010", "τ = 0.040"},
  AxesLabel → {"Input Current", "Spike Count (Hz)"},
  PlotMarkers → Automatic,
  PlotLabel → "f-I Curves for Different Time Constants"
]
```

Out[34]=



Experiment 6 Below: Frequency Encoding

Change the sine wave frequency from 5 Hz to 2 Hz, then 10 Hz, then 20 Hz. At what frequency

can the neuron no longer follow the oscillations? What does this tell you about the neuron's temporal resolution?

- Based on the results below, the neuron begins to fail to follow the oscillations at around 10–20 Hz, with 20 Hz showing the clearest breakdown. This tells you the neuron has a limited temporal resolution. It can represent slower changes in input, but once the signal changes too fast, the membrane potential cannot rise and fall quickly enough to track every cycle. In other words, the neuron behaves like a low-pass filter: slow inputs are preserved better than fast ones. The membrane time constant limits how quickly the neuron can respond, so high-frequency signals get smoothed out and are not encoded accurately in the spike train.

```
In[35]:= (* ===== *)
(* Comparison version: overlay all frequencies *)
(* ===== *)

Vrest = -65;
Vreset = -65;
Vth = -50;
Rm = 1;
tau = 0.02;

frequencies = {2, 5, 10, 20};

solutions = Table[
  Module[{Ifreq, sol},
    Ifreq[t_] := 15.0 + 12.0*Sin[2 Pi*f*t];
    sol = NDSolve[
      {
        V'[t] == -(V[t] - Vrest) + Rm*Ifreq[t]/tau,
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vth, V[t] → Vreset]
      },
      V,
      {t, 0, 1.0}
    ];
    {f, sol}
  ],
  {f, frequencies}
];

plotVAll = Plot[
  Evaluate[
```

```

Table[
  V[t] /. solutions[[i, 2]],
  {i, Length[solutions]}
],
{t, 0, 1.0},
PlotRange → All,
AxesLabel → {"Time (s)", "V (mV)"},
PlotLegends → Table[ToString[frequencies[[i]] <> " Hz", {i, Length[frequencies]}],
PlotLabel → "Membrane Potential for Different Input Frequencies"
];

```

```

plotIAll = Plot[
  Evaluate[
    Table[
      15.0 + 12.0*Sin[2 Pi*frequencies[[i]]*t],
      {i, Length[frequencies]}
    ]
  ],
  {t, 0, 1.0},
  PlotRange → All,
  AxesLabel → {"Time (s)", "I (nA)"},
  PlotLegends → Table[ToString[frequencies[[i]] <> " Hz", {i, Length[frequencies]}],
  PlotLabel → "Input Currents for Different Frequencies"
];

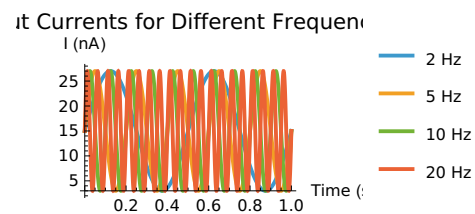
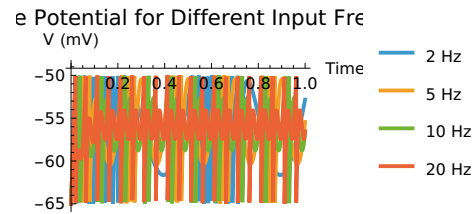
```

```

GraphicsColumn[{plotVAll, plotIAll}]

```

Out[44]=



EXPERIMENT REFLECTIONS

Experiment 1: f-I Curve

1. What parameter did you change, and to what values?

I changed the input current I from 0 nA to 120 nA in steps of 5 nA.

2. What did you expect to happen?

I expected that as the input current increased, the neuron would fire more often. A larger current should push the membrane potential upward faster, causing it to reach threshold more quickly and produce more spikes.

3. What actually happened?

The output showed that the spike count stayed near zero at very low currents, then increased steadily as current increased. The $f-I$ curve had an upward trend, showing that stronger input current led to more firing. My screenshot of the $f-I$ curve confirms this pattern.

4. Why does this make sense given the LIF equation?

In the LIF equation, the input current term adds to the membrane potential change: As I gets larger, dV/dt becomes larger, so the voltage rises faster toward threshold. This leads to more frequent spikes.

Experiment 2: Sine Wave Input at 5 Hz

1. What parameter did you change, and to what values?

I changed the input current from a constant value to a sine wave input:

$$I_{\text{sine}}(t) = 15 + 12 \sin(2\pi \cdot 5t)$$

This made the input oscillate between about 3 nA and 27 nA at 5 Hz.

2. What did you expect to happen?

I expected the neuron to fire more during the peaks of the sine wave, when the current was high, and fire less or stop during the troughs, when the current was low.

3. What actually happened?

That is exactly what happened. The spike train was clustered around the high parts of the sine wave, while fewer spikes appeared when the current dropped. The membrane potential plot and input current plot in my screenshot show that the neuron converted the continuous sine wave into bursts of spikes.

4. Why does this make sense given the LIF equation?

The sine wave changes the size of the input term $R_m I(t)$ over time. When the current is large, the membrane potential rises faster and reaches threshold more often. When the current is small, the voltage rises more slowly or falls back toward rest. This is why the spike rate tracks the changing input.

Experiment 3: Changing Sine Wave Frequency

1. What parameter did you change, and to what values?

I changed the frequency of the sine wave input from 5 Hz to 2 Hz, 10 Hz, and 20 Hz.

2. What did you expect to happen?

I expected the neuron to follow lower frequencies better because the membrane potential would have enough time to rise and fall with the input. At higher frequencies, I expected the neuron to struggle to keep up.

3. What actually happened?

The neuron followed 2 Hz best and still tracked 5 Hz reasonably well. At 10 Hz, the spike pattern became less clearly matched to the oscillation. At 20 Hz, the neuron could no longer follow the oscillations well, and the spike pattern no longer lined up clearly with each cycle of the input. My screenshot shows that the higher-frequency input waves became too fast for the membrane response to match.

4. Why does this make sense given the LIF equation?

The LIF model includes the membrane time constant, which determines how fast the voltage can respond. Because of this, the neuron acts like a low-pass filter: it can follow slow input changes, but it smooths out rapid ones. At high frequencies, the current changes direction too fast for the membrane potential to track every cycle, so the firing pattern no longer matches the input closely.

FINAL REFLECTIONS

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make “decisions”?

- As I gradually increased the input current from below threshold to above it, the neuron initially did not fire at all. The membrane potential increased slightly but stayed below the threshold. Once the current reached a certain level, the membrane potential crossed the threshold and the neuron began to produce spikes. As the current increased further, the spikes occurred more frequently.
- The neuron started firing when the input current was strong enough to push the membrane potential above the threshold voltage. This threshold behavior shows that neurons act like decision-making units. Instead of responding to every small input, they only produce an output spike when the combined input is strong enough. In biological neurons, this allows the brain to filter noise and only transmit meaningful signals.

2. Examine your f–I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

- The relationship in the f–I curve is not perfectly linear. At low currents, the neuron does not fire at all. Once the threshold is crossed, the firing rate begins to increase, and then it increases roughly steadily with higher current.
- This shape is important biologically because it shows how neurons encode stimulus intensity. Weak stimuli may produce no spikes, moderate stimuli produce some spikes, and strong stimuli produce higher firing rates. The brain can therefore represent the strength of a signal through spike rate, a process called rate coding.

3. How does changing the membrane time constant affect firing? In biological terms, what types of

neurons might have short vs. long time constants, and what would each be good at?

- Changing the membrane time constant changes how quickly the neuron responds to input. A short time constant means the membrane potential changes quickly, so the neuron can respond rapidly and fire more easily. A long time constant means the membrane changes more slowly, which smooths out fast inputs and leads to slower firing responses.
- In biological systems, neurons with short time constants might be involved in tasks that require fast responses, such as sensory processing or auditory neurons. Neurons with long time constants integrate signals over longer periods of time and may be useful for functions like decision-making or memory-related processing.

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss? (Think about: dendrite morphology, multiple ion channels, neurotransmitter dynamics, adaptation.)

- The LIF model captures some important features of real neurons. It represents how membrane potential changes over time, includes a threshold for firing, and resets after a spike. These properties allow it to reproduce basic spike timing and firing rate behavior.
- However, the model misses many biological details. Real neurons have complex dendritic structures that influence how signals are integrated. They also have many different ion channels that produce more complex spike shapes and dynamics. Additionally, real neurons involve neurotransmitter release, synaptic plasticity, and adaptation, which affect how neurons change their behavior over time. The LIF model simplifies these processes into a basic mathematical representation.

5. Wolfram's Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

- Using Wolfram's Manipulate[] made the learning process much clearer because I could instantly see how changing parameters affected the neuron's behavior. Instead of only reading about the equations, I could experiment with the system and observe the results in real time.
- The most surprising thing I discovered while working on this assignment was how suddenly the neuron begins firing once the threshold is crossed. Small increases in input current below the threshold had almost no effect, but once the threshold was reached, the neuron started producing spikes quickly. This helped me to better understand how neurons convert continuous inputs into discrete spike signals.